

WEEK2_SKILL

Session 3:

1. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_INPUT 1024
#define MAX_HISTORY 100

char history[MAX_HISTORY][MAX_INPUT];
int history_count = 0;
int history_index = 0;

void disable_raw_mode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

void enable_raw_mode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void add_history(char *command)
{
    if (strlen(command) == 0)
        return;
}
```

```

sanjay_dundra@Sanjay:~/shell_project$ nano shell.c
sanjay_dundra@Sanjay:~/shell_project$ gcc shell.c -o shell
sanjay_dundra@Sanjay:~/shell_project$ ./shell
shell> ls
Makefile  shell  shell.c
shell> ls
Makefile  shell  shell.c
shell> ls
Makefile  shell  shell.c
shell> pwd
/home/sanjay_dundra/shell_project
shell> whoami
sanjay_dundra
shell> date
Wed Aug 12 09:16:29 UTC 2026
shell> date |

```

Report: I compiled and executed the shell program and tested commands such as ls, pwd, whoami, and date. I observed that the shell successfully executed each command and displayed the correct output. Thus, I successfully tested the execution of multiple commands in the shell.

2. To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

```

sanjay_dundra@Sanjay:~/shell_project$ valgrind --version
valgrind-3.26.0
sanjay_dundra@Sanjay:~/shell_project$ █

```

```

sanjay_dundra@Sanjay:~/shell_project$ nano shell.c
sanjay_dundra@Sanjay:~/shell_project$ gcc -Wall -Wextra shell.c -o shell
sanjay_dundra@Sanjay:~/shell_project$ ./shell
shell> ls
Makefile  shell  shell.c
shell> pwd
/home/sanjay_dundra/shell_project
shell> whoami
sanjay_dundra
shell> date
Wed Aug 12 09:55:56 UTC 2026
shell> whoami █

```

Report: I compiled and executed the shell program and tested commands such as ls, pwd, whoami, and date. I observed that all the commands executed successfully and displayed the expected output. Thus, I successfully tested command execution in the shell.

```
sanjay_dundra@Sanjay:~/shell_project$ valgrind --leak-check=full --show-leak-kinds=all ./shell
==1328== Memcheck, a memory error detector
==1328== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==1328== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==1328== Command: ./shell
shell> ls
Makefile  shell  shell.c
shell> whoami
sanjay_dundra
shell> date
Wed Aug 12 09:58:29 UTC 2026
shell> whoami
sanjay_dundra
shell> exit
==1328==
==1328== HEAP SUMMARY:
==1328==   in use at exit: 0 bytes in 0 blocks
==1328==   total heap usage: 10 allocs, 10 frees, 1,174 bytes allocated
==1328==
==1328== All heap blocks were freed -- no leaks are possible
==1328==
==1328== For lists of detected and suppressed errors, rerun with: -s
==1328== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
sanjay_dundra@Sanjay:~/shell_project$
```

Report: I ran the shell using Valgrind and tested commands such as ls, whoami, and date. Valgrind reported 0 errors and confirmed that all allocated memory was freed with no memory leaks. Thus, I successfully verified that the shell is memory-safe.

Session 4:

1. To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```
sanjay_dundra@Sanjay:~/shell_project$ nano shell.c
sanjay_dundra@Sanjay:~/shell_project$ gcc -Wall -Wextra shell.c -o shell
sanjay_dundra@Sanjay:~/shell_project$ ./shell
shell> ls -l
total 36
-rw-r--r-- 1 sanjay_dundra sanjay_dundra  48 Aug 12 08:31 Makefile
-rwxr-xr-x 1 sanjay_dundra sanjay_dundra 17000 Aug 12 10:12 shell
-rw-r--r-- 1 sanjay_dundra sanjay_dundra  8200 Aug 12 10:12 shell.c
shell> █
```

Report: I implemented the tokenization of user input by splitting commands into individual tokens. I handled spaces and delimiters properly and created token structures for further processing. I also tested the output to make sure the tokens were generated correctly.

```
shell> ls;pwd
Makefile  shell  shell.c
/home/sanjay_dundra/shell_project
shell> ls | wc
      3      3     23
sanjay_dundra@Sanjay> |
```

Report: I tested multiple commands in the shell, including sequential commands `ls; pwd` and a pipeline `ls | wc`. Both executed successfully and produced the expected output. Thus, I successfully tested sequential command execution and command piping in the shell.

```
shell> ls |
sh: 1: Syntax error: end of file unexpected
shell> | ls
sh: 1: Syntax error: "|" unexpected
shell> ls ;;
sh: 1: Syntax error: ";;" unexpected
shell> ls -l
total 36
-rw-r--r-- 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay  48 Aug 12 08:31 Makefile
-rwxr-xr-x 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay 17000 Aug 12 10:12 shell
-rw-r--r-- 1 sanjay_dundra@Sanjay sanjay_dundra@Sanjay  8200 Aug 12 10:12 shell.c
shell> echo hello world
hello world
shell> |
```

Report: I tested invalid inputs such as incomplete pipes and separators, and the shell displayed appropriate syntax errors. I also tested `ls -l` and `echo hello world`, which executed successfully and produced the expected output.

Thus, I successfully tested error handling and normal command execution in the shell.

2. To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
shell>
Empty command - nothing to parse.

shell> |
```

Report: I tested without giving any command and it gave output as Empty Command

```
sanjay_dundra@Sanjay:~$ cd ~/shell_project
sanjay_dundra@Sanjay:~/shell_project$ nano shell.c
sanjay_dundra@Sanjay:~/shell_project$ gcc -Wall -Wextra shell.c -o shell
sanjay_dundra@Sanjay:~/shell_project$ ./shell
=====
                SIMPLE PARSER SHELL
=====
Type 'exit' to quit.

shell> ls -l

===== TOKENS =====
Token 1: ls
Token 2: -l
=====
Syntax: VALID

===== PARSE TREE =====
COMMAND
  +-- ls
  +-- -l
=====

===== EXECUTION STRUCTURE =====
Argument 1: ls
Argument 2: -l
=====
```

Report: I tested the parser shell with the command ls -l. The shell correctly identified the tokens, validated the syntax, generated the parse tree, and

displayed the execution structure with the command and its argument. Thus, I successfully verified the tokenization, parsing, and execution structure of the shell.

```
shell> ls | grep txt

===== TOKENS =====
Token 1: ls
Token 2: |
Token 3: grep
Token 4: txt
=====

Syntax: VALID

===== PARSE TREE =====
COMMAND
+-- ls
|
+-- PIPE
+-- grep
+-- txt
=====

===== EXECUTION STRUCTURE =====
Argument 1: ls
Operator: PIPE
Argument 3: grep
Argument 4: txt
=====
```

Report: I tested the parser with the command `ls | grep txt`. The shell correctly identified the tokens, validated the syntax, generated the parse tree with the pipe operator, and displayed the execution structure. Thus, I successfully verified pipe parsing and command structure handling.

```
shell> ls |

===== TOKENS =====
Token 1: ls
Token 2: |
=====

Syntax Error: Pipe cannot be last.
Command rejected.
```

Report: I tested the shell with an incomplete pipe command `ls |`. The parser correctly detected that the pipe was the last token and rejected the command with a syntax error. Thus, I successfully verified syntax error handling for incomplete pipe commands.